

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 1

GUÍA DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	<i>Tecnología de Objetos</i>				
TÍTULO DE LA PRÁCTICA:	<i>Laboratorio 03</i>				
NÚMERO DE PRÁCTICA:	3	AÑO LECTIVO:	2026	NRO. SEMESTRE:	VI
TIPO DE PRÁCTICA:	INDIVIDUAL				
	GRUPAL	MÍNIMO DE ESTUDIANTES	-----	MÁXIMO DE ESTUDIANTES	2
FECHA INICIO:	16/09/2026	FECHA FIN:	19/09/2026	DURACIÓN:	2 horas
INTEGRANTE (s): <i>Chipana Jeronimo German Arturo</i> <i>Baca Calsin Leonardo Juan José</i>					
DOCENTE(s): <i>Edith Giovanna Cano Mamani</i>					

I. SOLUCIÓN

Repositorio:

<https://github.com/ChipanaGerman/TecnologiaDeObjetos-Lab-B/tree/main/Lab3>

INTRODUCCIÓN

El presente informe detalla el diseño e implementación de un sistema de cálculo matemático concurrente centrado en la integración numérica mediante el método del trapecio. En el contexto de la ingeniería de software moderna, la eficiencia computacional es un requerimiento crítico, por lo que la resolución de problemas iterativos complejos exige el aprovechamiento de arquitecturas multinúcleo. Además de la resolución algorítmica orientada a objetos, este trabajo presenta un estudio comparativo empírico sobre cómo tres lenguajes con filosofías de diseño distintas (Java, C++ y Go) gestionan la concurrencia, el manejo dinámico de memoria y la orquestación de recursos mediante reserva de hilos (Thread Pools) en procesadores de 64 bits.

OBJETIVOS

Objetivo General

- Analizar y comparar la eficiencia de los modelos de programación paralela de los lenguajes Java, C++ y Go, implementando una solución concurrente para el cálculo de la integral definida de una función matemática mediante el método de aproximación del trapecio.

Objetivos Específicos

- Diseñar un algoritmo de integración numérica que particione el cálculo en sub-tareas independientes, iterando desde $N = 1$ particiones hasta alcanzar una convergencia absoluta basada en una tolerancia de punto flotante.
- Aplicar rigurosamente el paradigma de Programación Orientada a Objetos (POO) y la gestión de memoria dinámica para abstraer el motor de cálculo y la función polinomial objetivo.
- Implementar patrones de concurrencia eficientes (*Thread Pools* en Java, ejecución asíncrona nativa en C++ y *Goroutines* en Go) para minimizar el *overhead* del sistema operativo y evitar condiciones de carrera (Race Conditions).

MARCO CONCEPTUAL

- **Integración Numérica (Método del Trapecio):** Es una técnica de aproximación numérica que calcula el área bajo una curva dividiendo el intervalo de integración en N trapecios. Matemáticamente, a mayor número de trapecios (particiones más finas), el error de truncamiento disminuye, convergiendo asintóticamente hacia el valor exacto de la integral analítica.
- **Programación Paralela:** Es el paradigma de computación en el que múltiples instrucciones se ejecutan de forma simultánea en distintos núcleos de procesamiento. Su objetivo es dividir un problema grande en subproblemas independientes para reducir el tiempo total de ejecución (Speedup).

- **Patrón de Arquitectura “Thread Pool” (Reserva de Hilos):** En arquitecturas de 64 bits, instanciar y destruir hilos nativos del Sistema Operativo por cada tarea genera un *overhead* (costo de procesamiento y cambio de contexto) que puede superar el beneficio del paralelismo. Un *Thread Pool* resuelve esto manteniendo un grupo de hilos instanciados en estado de espera (idle). Cuando llegan tareas (como calcular un fragmento del área de la integral), se asignan a los hilos disponibles, reutilizándolos continuamente y evitando la saturación de memoria.

Problema Propuesto:

En Java se emplean interfaces funcionales, encapsulamiento en clases y el framework `java.util.concurrent` (`ExecutorService`, `Future`, `Callable`).

```
Java
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.*;

// 1. Abstracción de la Función Matemática (P00)
interface FuncionMatematica {
    double evaluar(double x);
}

// 2. Implementación concreta de  $f(x) = 2x^2 + 3x + 0.5$ 
class PolinomioCuadratico implements FuncionMatematica {
    @Override
    public double evaluar(double x) {
        return 2.0 * Math.pow(x, 2) + 3.0 * x + 0.5;
    }
}

// 3. Tarea concurrente para calcular un sub-rango de trapecios
class TareaTrapecio implements Callable<Double> {
    private final FuncionMatematica funcion;
    private final double a;
    private final double h;
    private final int inicio;
    private final int fin;

    public TareaTrapecio(FuncionMatematica funcion, double a, double h, int inicio, int fin) {
        this.funcion = funcion;
        this.a = a;
        this.h = h;
        this.inicio = inicio;
        this.fin = fin;
    }
}
```

```
@Override
public Double call() {
    double sumaParcial = 0.0;
    for (int i = inicio; i < fin; i++) {
        double x1 = a + i * h;
        double x2 = a + (i + 1) * h;
        sumaParcial += (funcion.evaluar(x1) + funcion.evaluar(x2)) * (h / 2.0);
    }
    return sumaParcial;
}
}

// 4. Clase orquestadora con Pool de Threads
class IntegradorParalelo {
    private final FuncionMatematica funcion;
    private final int numeroHilos;
    private final ExecutorService threadPool;

    public IntegradorParalelo(FuncionMatematica funcion) {
        this.funcion = funcion;
        this.numeroHilos = Runtime.getRuntime().availableProcessors();
        this.threadPool = Executors.newFixedThreadPool(this.numeroHilos);
    }

    public double calcularArea(double a, double b, int n) throws InterruptedException,
    ExecutionException {
        double h = (b - a) / n;
        int cantidadPorHilo = (int) Math.ceil((double) n / numeroHilos);

        List<Future<Double>> resultadosFuturos = new ArrayList<>();

        for (int t = 0; t < numeroHilos; t++) {
            int inicio = t * cantidadPorHilo;
            int fin = Math.min(inicio + cantidadPorHilo, n);

            if (inicio < n) {
                Callable<Double> tarea = new TareaTrapecio(funcion, a, h, inicio, fin);
                resultadosFuturos.add(threadPool.submit(tarea));
            }
        }

        double areaTotal = 0.0;
        for (Future<Double> resultado : resultadosFuturos) {
            areaTotal += resultado.get();
        }
    }
}
```

```
        return areaTotal;
    }

    public void cerrar() {
        this.threadPool.shutdown();
    }
}

// 5. Clase Principal
public class TrapecioIntegral {
    public static void main(String[] args) {
        double a = 2.0;
        double b = 20.0;
        double tolerancia = 1e-6; // Criterio de parada: cuando el valor se estabiliza

        FuncionMatematica funcion = new PolinomioCuadratico();
        IntegradorParalelo integrador = new IntegradorParalelo(funcion);

        System.out.println(" INTEGRACIÓN POR TRAPECIOS PARALELA (JAVA) ");
        System.out.printf("Intervalo: [%.2f, %.2f]\n", a, b);

        double areaAnterior = 0.0;
        int n = 1;
        boolean convergencia = false;

        try {
            while (!convergencia) {
                double areaActual = integrador.calcularArea(a, b, n);

                if (n > 1 && Math.abs(areaActual - areaAnterior) < tolerancia) {
                    System.out.printf("-> Convergencia alcanzada en n = %d trapecios.\n", n);
                    System.out.printf("Área final calculada: %.8f\n", areaActual);
                    convergencia = true;
                } else {
                    if (n == 1 || n % 1000 == 0) {
                        System.out.printf("Trapecios: %-8d | Área: %.8f\n", n, areaActual);
                    }
                    areaAnterior = areaActual;
                    n = (n < 64) ? n + 1 : n * 2; // Crecimiento progresivo para converger
                    eficientemente
                }
            }
        } catch (InterruptedException | ExecutionException e) {
            System.err.println("Error en ejecución concurrente: " + e.getMessage());
        } finally {
            integrador.cerrar();
        }
    }
}
```

```
}  
}  
}
```

Explicación Técnica de la Implementación

- **Arquitectura de Objetos:** La regla del polimorfismo se aplica mediante la interfaz `FuncionMatematica`, permitiendo desacoplar la función evaluada del algoritmo integrador.
- **Pool de Hilos:** Se implementa `Executors.newFixedThreadPool` dimensionado dinámicamente con `Runtime.getRuntime().availableProcessors()`. Esto evita la sobrecarga (*overhead*) de crear y destruir hilos a nivel del sistema operativo en cada iteración de `n`.
- **Sincronización y Futuros:** Cada sub-intervalo se calcula asíncronamente implementando `Callable<Double>`. Los resultados se devuelven en objetos `Future<Double>`, cuyo método `.get()` actúa como barrera de sincronización esperando el resultado de cada hilo para realizar la reducción final.

Ejecución:

```
PS C:\Users\User\German_UNSA\TERCER_ANIO\B\TECNOLOGIA_OBJETOS\Laboratorios\TecnologiaDeObjetos-Lab-B\Lab3> go run main.go  
INTEGRACIÓN POR TRAPECIOS PARALELA (GO)  
Intervalo: [2.00, 20.00] | Workers: 8  
Trapecios: 1 | Área: 7875.00000000  
-> Convergencia alcanzada en n = 131072 trapecios.  
Área final calculada: 5931.00000011
```

Versiones del code en C++ y en Go

❖ C++

El enunciado solicita explícitamente: "**Utilice C++ para gestionar la memoria dinámicamente**". Diseñamos una clase `ThreadPool` nativa desde cero con `std::thread`, sincronización con cerrojos `mutex` y variables de condición, junto con asignación dinámica de memoria (`new/delete` explícitos en heap para objetos de datos y tareas).

C/C++

```
#include <iostream>
```

```
#include <vector>
#include <cmath>
#include <thread>
#include <queue>
#include <mutex>
#include <condition_variable>
#include <future>
#include <functional>
#include <memory>

// 1. P00: Clase base abstracta para la función
class FuncionBase {
public:
    virtual ~FuncionBase() = default;
    virtual double evaluar(double x) const = 0;
};

// Implementación de  $f(x) = 2x^2 + 3x + 0.5$ 
class PolinomioCuadratico : public FuncionBase {
public:
    double evaluar(double x) const override {
        return 2.0 * x * x + 3.0 * x + 0.5;
    }
};

// 2. Estructura asignada dinámicamente para los rangos de cálculo
struct RangoCalculo {
    int inicio;
    int fin;
    double h;
    double a;

    RangoCalculo(int i, int f, double delta, double limiteInf)
        : inicio(i), fin(f), h(delta), a(limiteInf) {}
};

// 3. Implementación de un Thread Pool orientado a objetos con C++ moderno
class ThreadPool {
private:
    std::vector<std::thread> trabajadores;
    std::queue<std::function<void()>> tareas;
    std::mutex colaMutex;
    std::condition_variable condicion;
    bool detener;

public:
    ThreadPool(size_t numHilos) : detener(false) {
        for (size_t i = 0; i < numHilos; ++i) {
            trabajadores.emplace_back([this]() {
                while (true) {
                    std::function<void()> tarea;
                    {
                        std::unique_lock<std::mutex> lock(this->colaMutex);
```

```
        this->condicion.wait(lock, [this]() {
            return this->detener || !this->tareas.empty();
        });
        if (this->detener && this->tareas.empty()) {
            return;
        }
        tarea = std::move(this->tareas.front());
        this->tareas.pop();
    }
    tarea();
}
});
}

template<class F>
auto encolar(F&& f) -> std::future<typename std::result_of<F()>::type> {
    using tipoRetorno = typename std::result_of<F()>::type;
    auto tarea = std::make_shared<std::packaged_task<tipoRetorno>>>(std::forward<F>(f));
    std::future<tipoRetorno> res = tarea->get_future();
    {
        std::unique_lock<std::mutex> lock(colaMutex);
        if (detener) {
            throw std::runtime_error("No se pueden encolar tareas en un ThreadPool detenido");
        }
        tareas.emplace([tarea]() { (*tarea)(); });
    }
    condicion.notify_one();
    return res;
}

~ThreadPool() {
    {
        std::unique_lock<std::mutex> lock(colaMutex);
        detener = true;
    }
    condicion.notify_all();
    for (std::thread &trabajador : trabajadores) {
        if (trabajador.joinable()) {
            trabajador.join();
        }
    }
}

};

// 4. Clase Integrador Numérico con Gestión Dinámica de Memoria
class IntegradorTrapecio {
private:
    const FuncionBase* funcion; // Referencia a objeto en Heap
    ThreadPool pool;
    size_t nucleos;

public:
```

```
IntegradorTrapezio(const FuncionBase* fn)
: funcion(fn),
  nucleos(std::thread::hardware_concurrency()),
  pool(nucleos > 0 ? nucleos : 4) {}

double integrar(double a, double b, int n) {
    double h = (b - a) / n;
    int paso = std::ceil((double)n / nucleos);
    std::vector<std::future<double>> futuros;

    // Gestión de memoria dinámica manual en heap para los descriptores de tarea
    std::vector<RangoCalculo*> buffersRangos;

    for (size_t t = 0; t < nucleos; ++t) {
        int inicio = t * paso;
        int fin = std::min(inicio + paso, n);

        if (inicio < n) {
            // Asignación dinámica explícita (Heap)
            RangoCalculo* rango = new RangoCalculo(inicio, fin, h, a);
            buffersRangos.push_back(rango);

            futuros.push_back(pool.encolar([this, rango]() -> double {
                double sumaLocal = 0.0;
                for (int i = rango->inicio; i < rango->fin; ++i) {
                    double x1 = rango->a + i * rango->h;
                    double x2 = rango->a + (i + 1) * rango->h;
                    sumaLocal += (this->funcion->evaluar(x1) + this->funcion->evaluar(x2)) *
(rango->h / 2.0);
                }
                return sumaLocal;
            }));
        }
    }

    double areaTotal = 0.0;
    for (auto &fut : futuros) {
        areaTotal += fut.get();
    }

    // Liberación explícita de memoria dinámica para evitar fugas (Memory Leaks)
    for (RangoCalculo* ptr : buffersRangos) {
        delete ptr;
    }

    return areaTotal;
}

int main() {
    double a = 2.0;
    double b = 20.0;
    double tolerancia = 1e-6;
```

```
// Instanciación dinámica del objeto Polinomio en el Heap (Memoria dinámica)
FuncionBase* polinomio = new PolinomioCuadratico();
IntegradorTrapezio* integrador = new IntegradorTrapezio(polinomio);

std::cout << " INTEGRACIÓN POR TRAPECIOS PARALELA (C++) " << std::endl;
std::cout << "Intervalo: [" << a << ", " << b << "]" << std::endl;

double areaAnterior = 0.0;
int n = 1;
bool convergencia = false;

while (!convergencia) {
    double areaActual = integrador->integrar(a, b, n);

    if (n > 1 && std::abs(areaActual - areaAnterior) < tolerancia) {
        std::cout << "-> Convergencia alcanzada en n = " << n << " trapezios." << std::endl;
        std::cout.precision(10);
        std::cout << "Área final calculada: " << std::fixed << areaActual << std::endl;
        convergencia = true;
    } else {
        if (n == 1 || n % 1000 == 0) {
            std::cout << "Trapezios: " << n << " | Área: " << areaActual << std::endl;
        }
        areaAnterior = areaActual;
        n = (n < 64) ? n + 1 : n * 2;
    }
}

// Limpieza de memoria dinámica
delete integrador;
delete polinomio;

return 0;
}
```

- **Gestión Dinámica de Memoria:** Se implementa el ciclo de vida en memoria libre (*Heap*) mediante operadores `new` y `delete`. Tanto las estructuras `RangoCalculo` que parametrizan la tarea de cada hilo como las instancias del motor de integración son reservadas y liberadas explícitamente, garantizando cero *memory leaks*.
- **Thread Pool en C++:** Como C++ estándar no incluye una clase `ThreadPool` integrada en la STL, se construye una cola protegida por `std::mutex` y sincronizada por `std::condition_variable`. Los hilos se suspenden cuando no hay trabajo y se activan selectivamente cuando ingresan tareas con `notify_one()`.

- **Empaquetado de Tareas:** Se utiliza `std::packaged_task` y `std::future`, permitiendo desacoplar la ejecución del hilo y recolectar el valor devuelto (`double`) de forma asíncrona y segura contra carreras de datos (*Data Races*).

INTEGRACIÓN POR TRAPÉCIOS PARALELA (C++)

Intervalo: [2, 20]

Trapezios: 1 | Área: 7875

-> Convergencia alcanzada en $n = 131072$ trapezios.

Área final calculada: 5931.0000001133

❖ Go

Go no utiliza threads del SO de manera directa; utiliza **Goroutines** (hilos lógicos ultraligeros en espacio de usuario). El patrón nativo para simular e implementar un *Thread/Worker Pool* de alto rendimiento es mediante canales con búfer (buffered channels) y `sync.WaitGroup`.

```
Go
package main

import (
    "fmt"
    "math"
    "runtime"
    "sync"
)

// 1. P00 en Go mediante Interfaces y Estructuras
type FuncionMatematica interface {
    Evaluar(x float64) float64
}

// Estructura que implementa la interfaz
type PolinomioCuadratico struct{}

func (p PolinomioCuadratico) Evaluar(x float64) float64 {
    return 2.0*math.Pow(x, 2) + 3.0*x + 0.5
}

// 2. Estructura de Trabajo para el Worker Pool
type Tarea struct {
    Inicio int
    Fin    int
    A      float64
    H      float64
}
```

```
}
```

```
// 3. Worker del Pool: Ejecuta concurrentemente sobre Goroutines
```

```
func worker(id int, funcion FuncionMatematica, tareas <-chan Tarea, resultados chan<- float64, wg
*sync.WaitGroup) {
    defer wg.Done()
    for tarea := range tareas {
        sumaLocal := 0.0
        for i := tarea.Inicio; i < tarea.Fin; i++ {
            x1 := tarea.A + float64(i)*tarea.H
            x2 := tarea.A + float64(i+1)*tarea.H
            sumaLocal += (funcion.Evaluar(x1) + funcion.Evaluar(x2)) * (tarea.H / 2.0)
        }
        resultados <- sumaLocal
    }
}
```

```
// 4. Integrador que administra el Pool de Goroutines
```

```
type IntegradorGo struct {
    Funcion    FuncionMatematica
    NumWorkers int
}

func NuevoIntegrador(fn FuncionMatematica) *IntegradorGo {
    return &IntegradorGo{
        Funcion:    fn,
        NumWorkers: runtime.NumCPU(),
    }
}

func (ig *IntegradorGo) Integrar(a, b float64, n int) float64 {
    h := (b - a) / float64(n)
    paso := int(math.Ceil(float64(n) / float64(ig.NumWorkers)))

    tareas := make(chan Tarea, ig.NumWorkers)
    resultados := make(chan float64, ig.NumWorkers)
    var wg sync.WaitGroup

    // Despacho de Workers fijos (Pool)
    for w := 0; w < ig.NumWorkers; w++ {
        wg.Add(1)
        go worker(w, ig.Funcion, tareas, resultados, &wg)
    }

    // Envío de fragmentos de integración a los canales
    for t := 0; t < ig.NumWorkers; t++ {
        inicio := t * paso
        fin := inicio + paso
        if fin > n {
            fin = n
        }
        if inicio < n {
            tareas <- Tarea{Inicio: inicio, Fin: fin, A: a, H: h}
        }
    }
}
```

```
    }  
}  
close(tareas) // Cierra el canal de tareas para que los workers finalicen el bucle  
  
// Goroutine auxiliar para cerrar el canal de resultados cuando terminen todos los workers  
go func() {  
    wg.Wait()  
    close(resultados)  
}()  
  
// Recolección y reducción del área total  
areaTotal := 0.0  
for res := range resultados {  
    areaTotal += res  
}  
  
return areaTotal  
}  
  
func main() {  
    a := 2.0  
    b := 20.0  
    tolerancia := 1e-6  
  
    funcion := PolinomioCuadratico{}  
    integrador := NuevoIntegrador(funcion)  
  
    fmt.Println(" INTEGRACIÓN POR TRAPECIOS PARALELA (GO) ")  
    fmt.Printf("Intervalo: [%.2f, %.2f] | Workers: %d\n", a, b, integrador.NumWorkers)  
  
    areaAnterior := 0.0  
    n := 1  
    convergencia := false  
  
    for !convergencia {  
        areaActual := integrador.Integrar(a, b, n)  
  
        if n > 1 && math.Abs(areaActual-areaAnterior) < tolerancia {  
            fmt.Printf("-> Convergencia alcanzada en n = %d trapecios.\n", n)  
            fmt.Printf("Área final calculada: %.8f\n", areaActual)  
            convergencia = true  
        } else {  
            if n == 1 || n%1000 == 0 {  
                fmt.Printf("Trapecios: %-8d | Área: %.8f\n", n, areaActual)  
            }  
            areaAnterior = areaActual  
            if n < 64 {  
                n++  
            } else {  
                n *= 2  
            }  
        }  
    }  
}
```

}

- **Modelo CSP (Communicating Sequential Processes):** Go no comparte memoria para comunicarse; usa canales (*chan*). El pool de workers se abastece de un canal con búfer tareas y escribe en el canal resultados.
- **Goroutines frente a Threads del SO:** Un hilo del SO en Java/C++ consume aproximadamente 1 MB de stack al inicio. Una Goroutine en Go inicia con solo 2 KB a 4 KB de stack dinámico. El planificador interno en modo M:N multiplexa miles de Goroutines sobre pocos hilos del procesador sin sobrecargar el kernel.
- **Sincronización Limpia:** *sync.WaitGroup* coordina la finalización de los *workers* sin necesidad de *mutexes* o variables de condición manuales, reduciendo notablemente el riesgo de interbloqueos (*deadlocks*).

```
PS C:\Users\User\German_UNSA\TERCER_ANIO\B\TECNOLOGIA_OBJETOS\Laboratorios\TecnologiaDeObjetos-Lab-B\Lab3> go run main.go
INTEGRACIÓN POR TRAPECIOS PARALELA (GO)
Intervalo: [2.00, 20.00] | Workers: 8
Trapezios: 1 | Área: 7875.00000000
-> Convergencia alcanzada en n = 131072 trapezios.
Área final calculada: 5931.00000011
```

II. CUESTIONARIO

1. ¿Cuál de los LP posee ventajas para programar paralelamente?

Desde una perspectiva arquitectónica y de ingeniería de software, Go (Golang) presenta las mayores ventajas nativas para la concurrencia moderna. Su modelo se basa en *Goroutines*, las cuales son multiplexadas dinámicamente sobre hilos del sistema operativo (modelo M:N) a través del *Go Scheduler*. Esto permite lanzar miles de tareas paralelas con un consumo de memoria inicial mínimo (aproximadamente 2 KB por tarea), superando la limitación estructural de memoria que sufren los hilos tradicionales. Además, la sincronización se maneja mediante el paso de mensajes (*Channels*), lo que reduce drásticamente los bloqueos complejos (uso de *mutexes*) y las condiciones de carrera (*race conditions*) comunes en los modelos de memoria compartida.

Por otro lado:

- **Java:** Ofrece el ecosistema empresarial más maduro y seguro mediante su API `java.util.concurrent`. Aunque tradicionalmente sus hilos tienen una relación 1:1 con el sistema operativo (lo que los hace

pesados, con ~1MB por hilo), herramientas de alto nivel como `ExecutorService` estandarizan la gestión de *Pools* y abstraen gran parte de la complejidad de la orquestación de hilos.

- C++: Otorga el control más granular sobre el procesador, las cachés y la memoria dinámica, logrando el mayor rendimiento posible en tiempo de ejecución. Sin embargo, su curva de complejidad es la más alta. Delega enteramente en el desarrollador la responsabilidad de evitar interbloqueos (*deadlocks*) y sincronizar recursos, careciendo además de abstracciones de *Thread Pools* nativas y directas en su biblioteca estándar, requiriendo implementaciones manuales o bibliotecas externas.

2. ¿Para cada lenguaje elabore una tabla cuando usa Pool de Treads?

Criterio Técnico	Java (<code>ExecutorService</code>)	C++ (<code>std::thread</code> / <code>Custom Pool</code>)	Go (<code>Goroutines</code> / <code>Channels</code>)
Relación con el Sistema Operativo	1:1 (Cada hilo de la JVM equivale a un hilo nativo del kernel).	1:1 (Interacción directa con la API de hilos POSIX o de Windows).	M:N (Múltiples <i>Goroutines</i> se ejecutan sobre un número menor de hilos nativos).
Costo de Memoria por Tarea	Alto (Reserva un bloque de pila predeterminado de ~1 MB por hilo).	Alto (Depende del compilador, pero típicamente requiere de 1 a 2 MB por hilo).	Mínimo (Utiliza una pila dinámica que inicia en aproximadamente ~2 KB).
Gestión y Creación del Pool	Nativamente integrada en el framework (factorías como <code>Executors</code>).	No estandarizada; requiere construir implementaciones manuales concurrentes o usar <code>std::async</code> .	Innecesaria; el planificador interno distribuye la carga automáticamente sin requerir <i>Pools</i> clásicos.

**Penalización por
Cambio de Contexto**

Moderada a Alta (El cambio de contexto invoca al Kernel del Sistema Operativo).

Moderada a Alta (Sufre la misma latencia por llamadas al sistema).

Muy Baja (El cambio se resuelve internamente en el espacio de usuario, sin invocar al Kernel).

III. CONCLUSIONES Y RETROALIMENTACIÓN GENERAL

Conclusiones del Laboratorio

- La comparativa entre los tres lenguajes demuestra que la arquitectura del modelo de concurrencia impacta directamente en el rendimiento y la facilidad de implementación. Go destaca al abstraer los hilos del sistema operativo mediante goroutines y canales, lo que permite dividir la suma de trapecios con un consumo de memoria sumamente bajo y sin lidiar con bloqueos manuales. Por el contrario, tanto Java como C++ dependen de hilos nativos más pesados, requiriendo un diseño riguroso con Thread Pools para evitar la sobrecarga por creación de recursos.
- El uso del patrón Thread Pool resultó indispensable para resolver la integración numérica masiva sin degradar el tiempo de ejecución. Al procesar hasta 131,072 particiones para alcanzar la convergencia deseada, repartir los subintervalos entre un número fijo de trabajadores reutilizables evitó la penalización por cambios de contexto a nivel de kernel. Esto confirma que, en problemas computacionales iterativos, la reutilización de recursos de ejecución es tan determinante como la propia precisión del método matemático.
- Existe una compensación directa entre el control de bajo nivel y la productividad del desarrollador. C++ brinda el máximo control sobre la memoria libre (heap) y la afinidad del procesador, pero exige construir la infraestructura de sincronización manualmente y asumir el riesgo de fugas de memoria. Java equilibra este panorama ofreciendo un framework robusto y seguro a través de ExecutorService, mientras que Go entrega el balance más eficiente e intuitivo al integrar concurrencia ligera de forma nativa en el lenguaje.

Retroalimentación General



UNIVERSIDAD NACIONAL DE SAN AGUSTIN
FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA



Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación

Aprobación: 2022/03/01

Código: GUIA-PRLD-001

Página: 17

IV. REFERENCIAS